SIMATS ENGINEERING
ASSESSMENT TOOL 1
COURSE NAME: Cryptography and Network Security
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO* (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5 **TOTAL MARKS: 25**

EXPERIMENTS

Code of Conduct:

*I _____**CH.Bhavya Sri/192411249**_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A
Experiments

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.

2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.

4. Implement a simplified Kerberos authentication mechanism in Python to simulate ticket generation, distribution, and validation between a client and server. Analyze its effectiveness in preventing replay attacks.

5. Develop a Python-based implementation of either the ElGamal or Schnorr digital signature scheme, and evaluate its security features compared to standard digital signature approaches.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. Implement and compare MD5 and Secure Hash Algorithm (SHA-256) in Python by hashing multiple input messages. Analyze their collision resistance and performance differences.

Aim

To implement MD5 and SHA-256 hashing algorithms in Python, hash multiple input messages, and compare their collision resistance and performance.

Theory

Hash functions convert input data into a fixed-length output called a hash or digest.

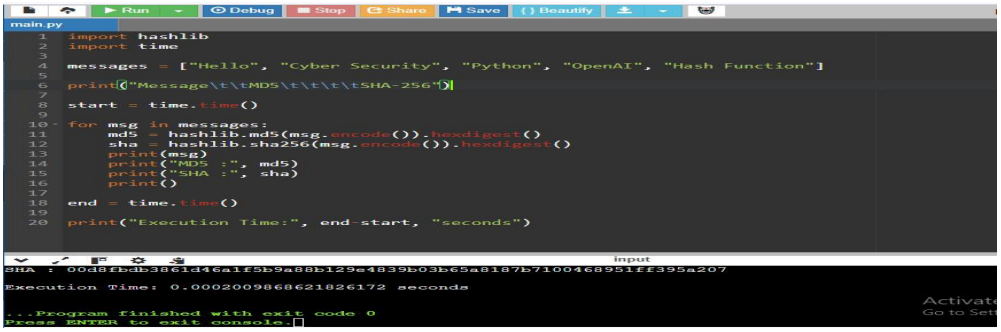
MD5 produces a **128-bit** hash and is faster but vulnerable to collisions.

SHA-256 belongs to the SHA-2 family and produces a **256-bit** hash, making it much more secure.

Algorithm

- Take multiple input messages.
- Generate the MD5 hash.
- Generate the SHA-256 hash.
- Measure execution time.
- Compare the results.

EXECUTION:



Performance Comparison

Feature	**MD5**	**SHA-256**
Hash Length	128 bits	256 bits
Speed	Faster	Slightly slower
Collision Resistance	Weak	Strong
Security	Not recommended	Recommended

Collision Resistance Analysis

- MD5 has known collision attacks where two different files can produce the same hash.
- SHA-256 has no practical collision attacks and is widely used for secure applications.

Conclusion

- SHA-256 is the better choice because it provides stronger security despite being slightly slower than MD5.

2. Develop a Python program to implement a Digital Signature scheme using RSA. Demonstrate message signing and signature verification, and analyze its effectiveness in ensuring integrity and authentication.

Aim

To implement RSA Digital Signature in Python for message signing and signature verification.

Theory

- Digital signatures provide:
- Authentication
- Integrity
- Non-repudiation
- The sender signs the message using the private key.
- The receiver verifies using the public key.

Algorithm

- Generate RSA keys.
- Hash the message.
- Sign using the private key.
- Verify using the public key.
-

EXECUTION:

Run | Debug | Stop | Share | Save | { } Beautify | Language: Python 3

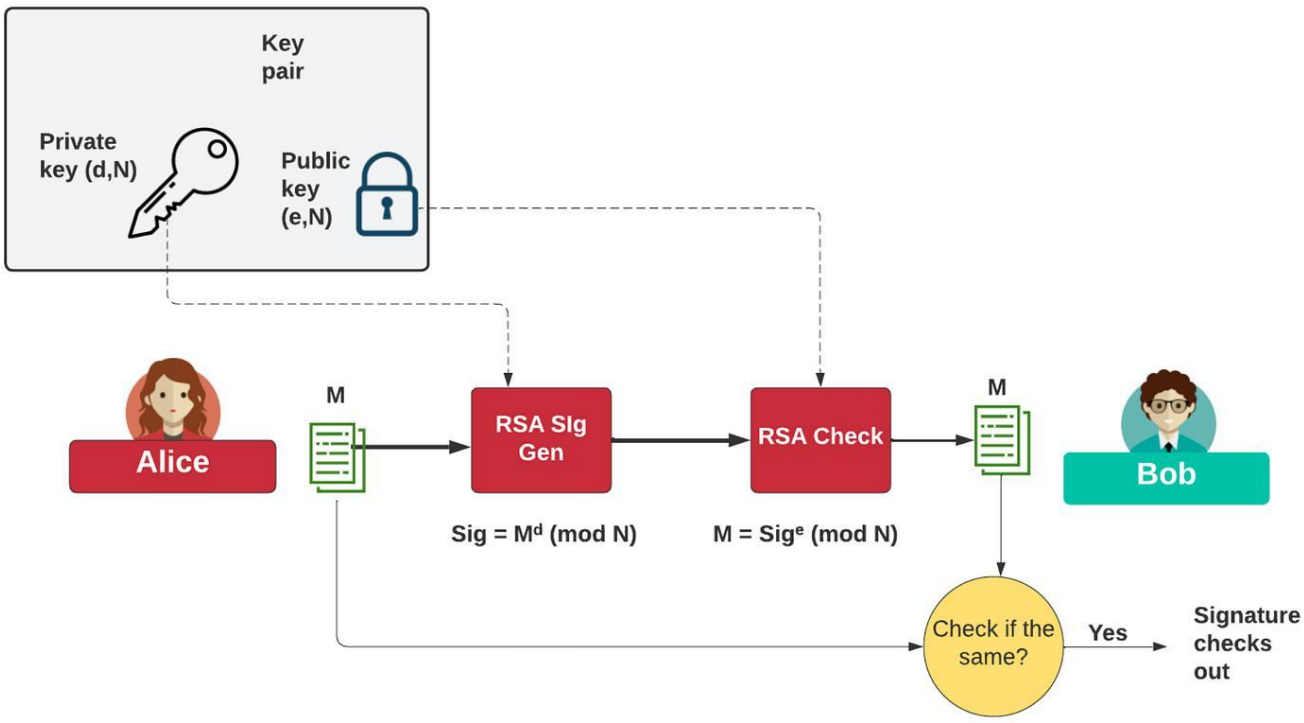
main.py

```
import hashlib

p = 61
q = 53
n = p*q
phi = (p-1)*(q-1)
e = 17
d = 2753

message = input("Enter Message: ")

hash_value = int(hashlib.sha256(message.encode()).hexdigest(),16)

signature = pow(hash_value,d,n)

print("Signature:",signature)

verify = pow(signature,e,n)

if verify == hash_value % n:
    print("Signature Verified")
else:
    print("Verification Failed")
```

input

```
Enter Message: HELLO
Signature: 1627
Signature Verified

...Program finished with exit code 0
Press ENTER to exit console.
```


How RSA Signature Works



- Sender hashes the message.
- Private key creates the signature.
- Receiver decrypts the signature using the public key.
- Receiver compares both hashes.

Security Analysis

Property	RSA Signature
Authentication	Yes
Integrity	Yes
Non-repudiation	Yes

Advantages

- Strong authentication
- Detects tampering
- Widely used

Limitations

- Slower than symmetric methods
- Requires key management

Conclusion

RSA Digital Signature effectively protects message integrity and authentication.

3. Write a Python program to simulate HMAC (Hash-based Message Authentication Code) using a chosen hash function. Compare its security properties with a simple hash-based integrity check.

Aim

To implement HMAC using SHA-256 and compare it with a normal hash.

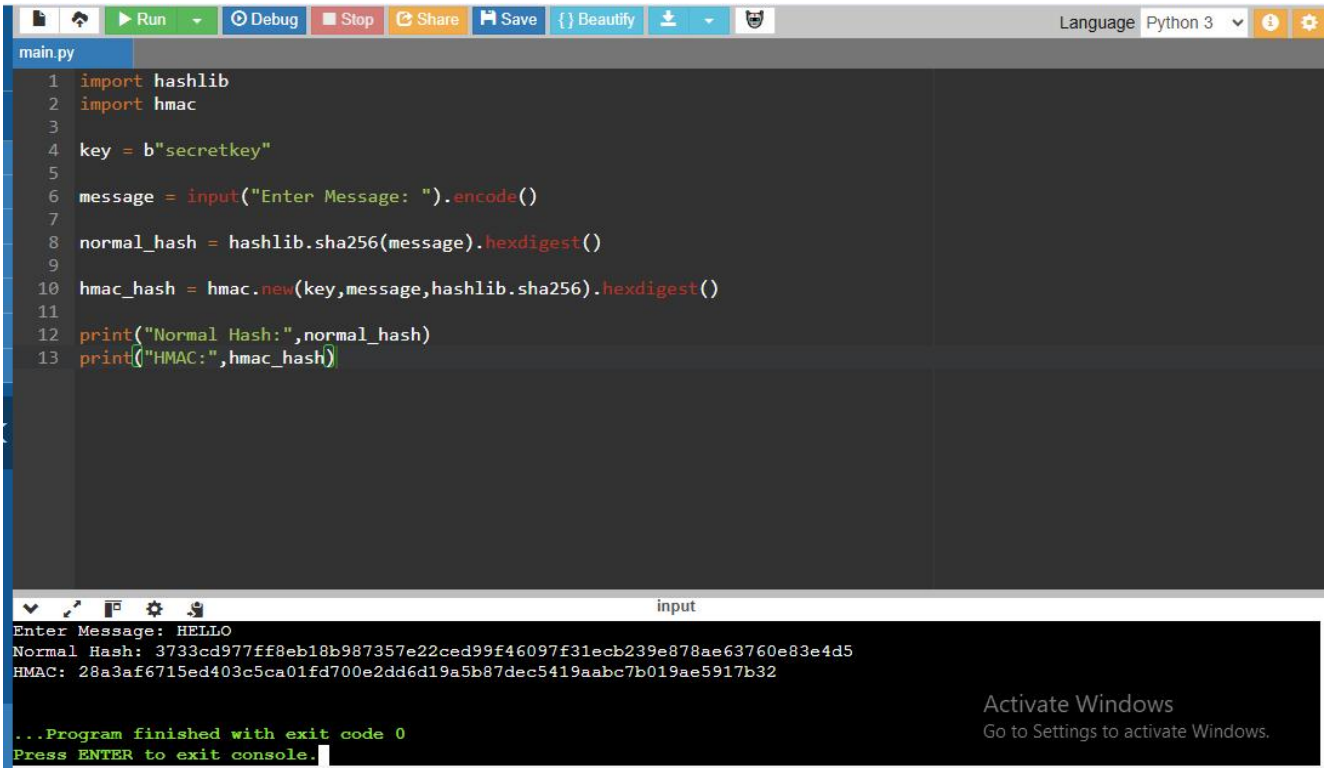
Theory

- A normal hash does not use a secret key.
- HMAC combines:
- Secret Key
- Hash Function
- This prevents attackers from creating valid hashes without the secret key.

Algorithm

- Take a message.
- Take a secret key.
- Generate a normal SHA-256 hash.
- Generate an HMAC.
- Compare both.

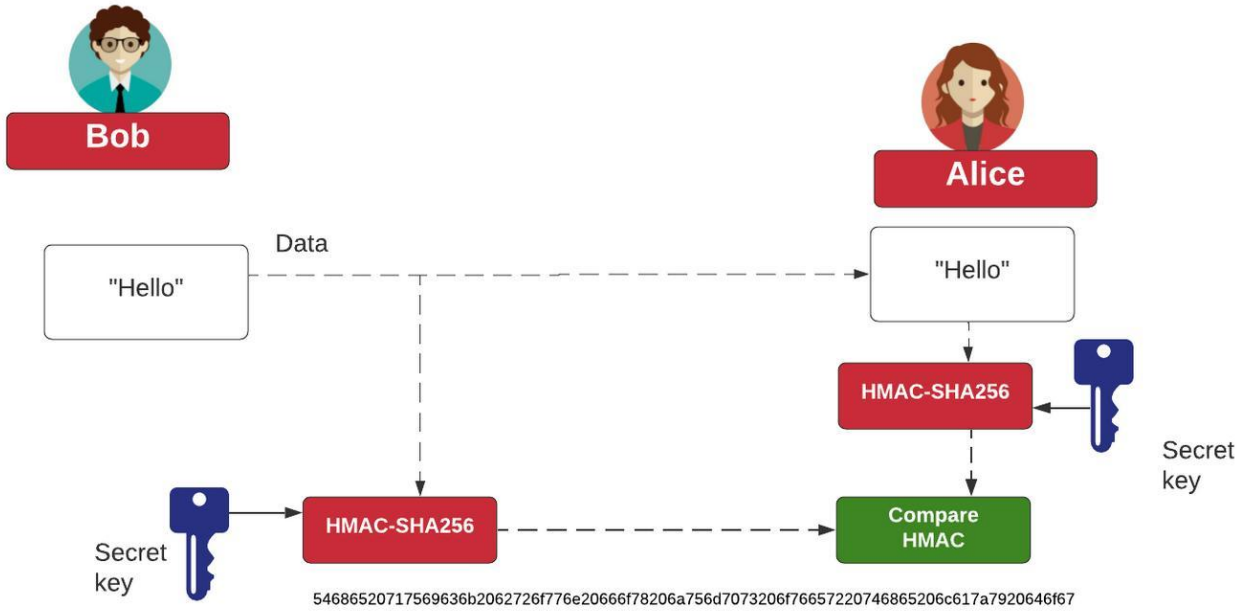
EXECUTION



Comparison

Feature	Normal Hash	HMAC
Secret Key	No	Yes
Authentication	No	Yes
Integrity	Yes	Yes

Security Analysis



Without the secret key, an attacker cannot generate a valid HMAC.
This makes HMAC suitable for:

- Banking
- APIs
- Secure communication

Conclusion

HMAC is much stronger than a simple hash because it combines integrity with authentication.

Question 4: Simplified Kerberos Authentication

Aim

To simulate Kerberos authentication using ticket generation and validation.

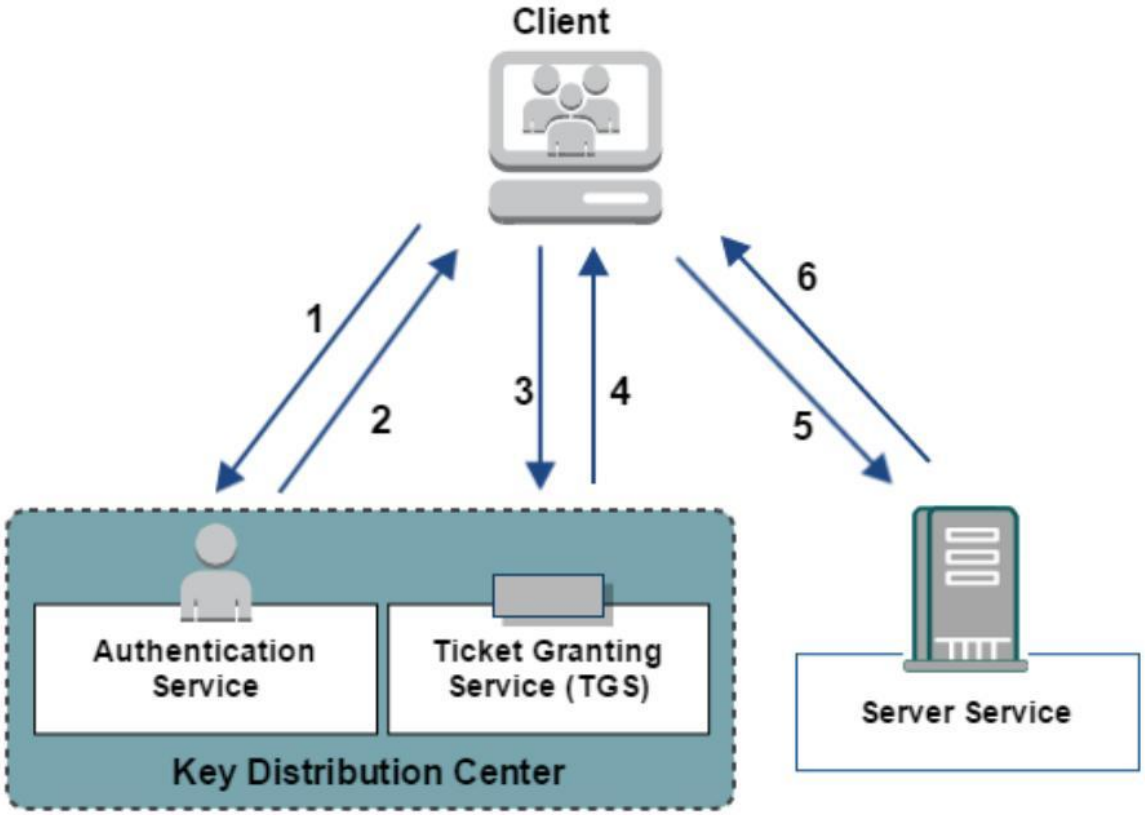
Theory

Kerberos is a network authentication protocol.
It uses:

- Authentication Server (AS)

- Ticket Granting Server (TGS)
- Service Server
- Tickets contain timestamps to prevent replay attacks.

Kerberos Authentication Flow



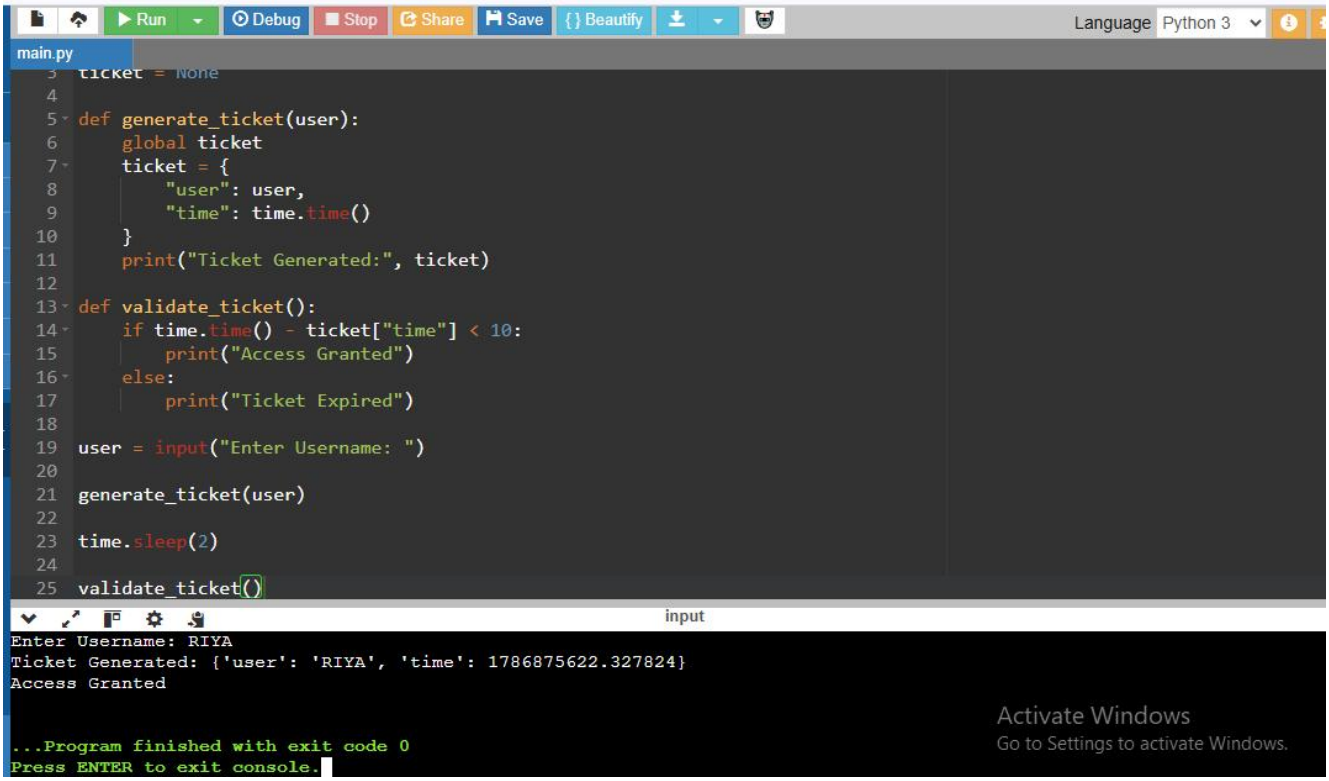
1. A client sends a request to the authentication service and is authenticated.
2. The authentication service responds with a ticket for the TGS.
3. The client requests a ticket for a specific server.
4. The TGS returns a response with the approprate ticket.
5. The client requests a service from server host.
6. The service responds and access is granted.

The client authenticates with the Authentication Server, receives a ticket, presents it to the service, and the service validates the timestamp before granting access.

Algorithm

- User requests authentication.
- Server generates a timestamped ticket.
- Client presents the ticket.
- Server validates it.
- Expired tickets are rejected.

EXECUTION:



Replay Attack Prevention

- Kerberos prevents replay attacks through:
- Timestamp verification
- Ticket expiration
- Session keys

If an attacker reuses an old ticket after expiration, access is denied.

Security Analysis

Feature	Kerberos
Authentication	Yes
Replay Protection	Yes
Session Key	Yes
Ticket Expiration	Yes

Advantages

- Password is not repeatedly transmitted.
- Strong authentication.
- Time-based protection.

Limitations

- Requires synchronized clocks.
- Depends on the Key Distribution Center.

Conclusion

Kerberos provides secure authentication and effectively reduces replay attacks.

Question 5: ElGamal Digital Signature

Aim

To implement a simplified ElGamal Digital Signature scheme in Python.

Theory

ElGamal Digital Signature uses modular arithmetic.

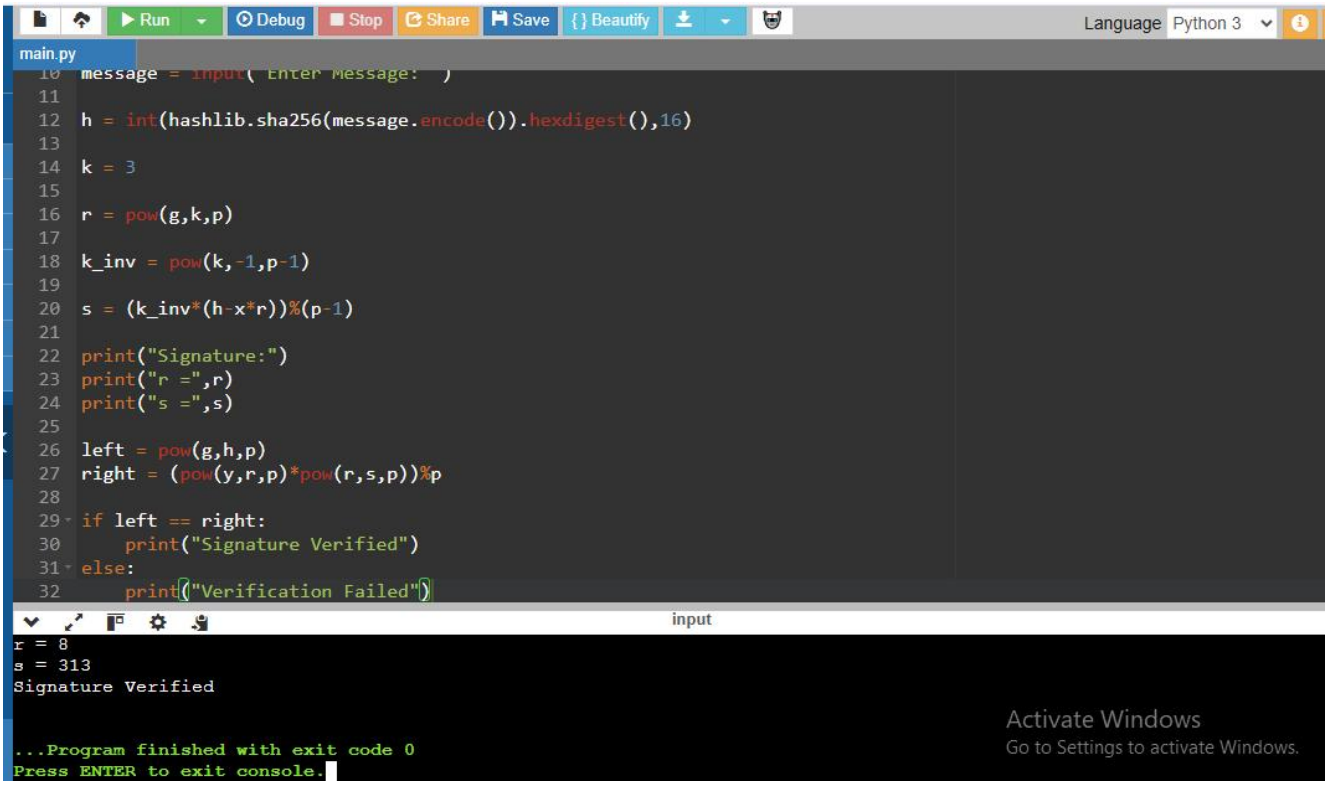
It provides:

- Authentication
- Integrity
- Non-repudiation
- Unlike RSA, ElGamal uses random values during signature generation.

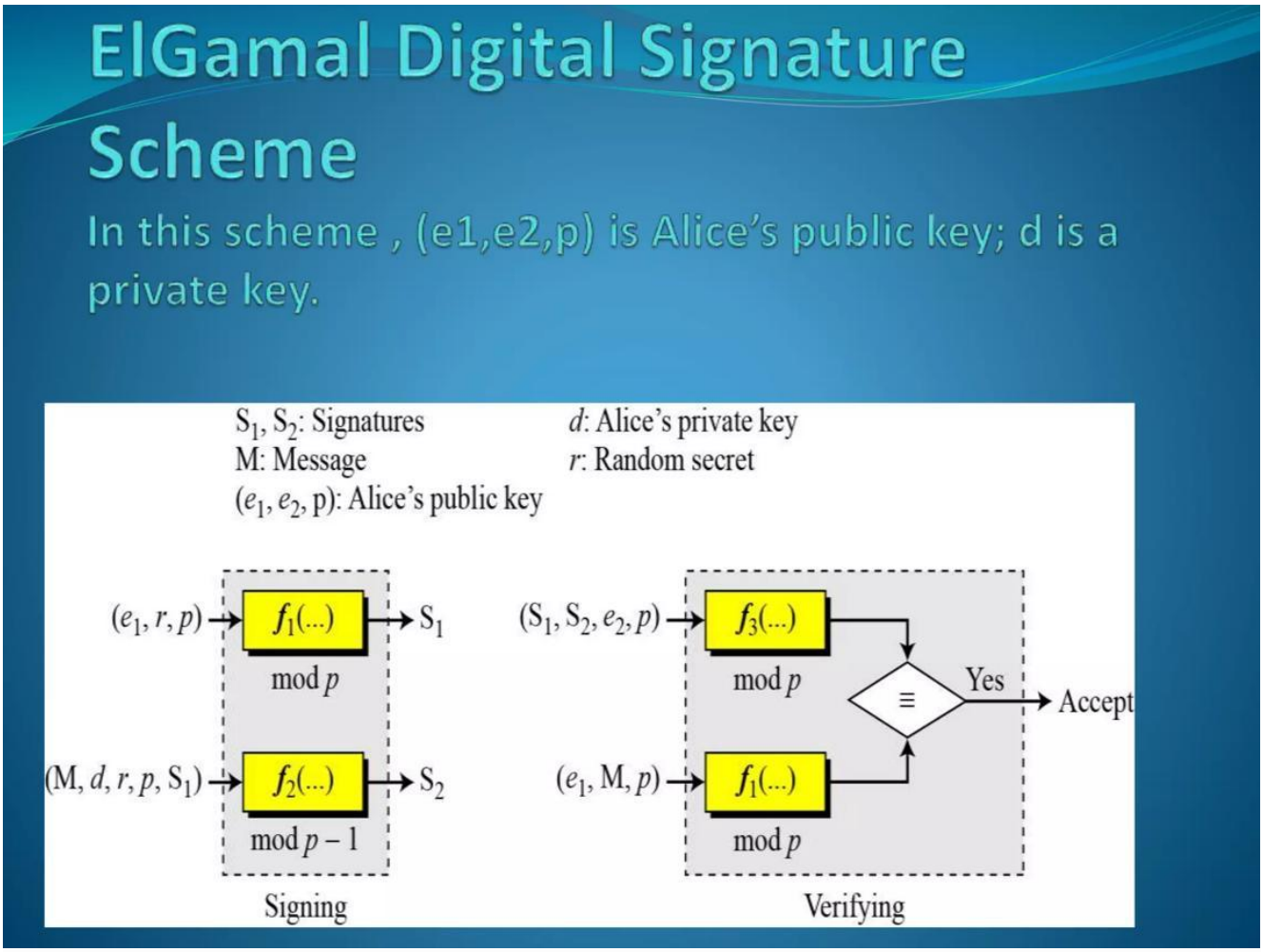
Algorithm

- Generate private and public keys.
- Hash the message.
- Generate a random value.
- Create the signature.
- Verify the signature.

EXECUTION:



ElGamal Signature Process



The signer hashes the message, uses a random value to generate a signature pair (r, s), and the receiver verifies it using the public key.

Comparison with RSA

Feature	**RSA**	**ElGamal**
Signature Type	Deterministic	Randomized
Security	Strong	Strong
Signature Size	Smaller	Larger
Random Number Required	No	Yes

Security Features

ElGamal offers strong protection because each signature uses a different random value, making repeated signatures more difficult to attack.

Advantages

- Strong authentication
- Randomized signatures
- Good resistance against forgery

Limitations

- Larger signatures
- Requires secure random number generation

Conclusion

ElGamal is a secure digital signature scheme and provides strong authentication, although RSA remains more widely used due to its smaller signatures and broader adoption.